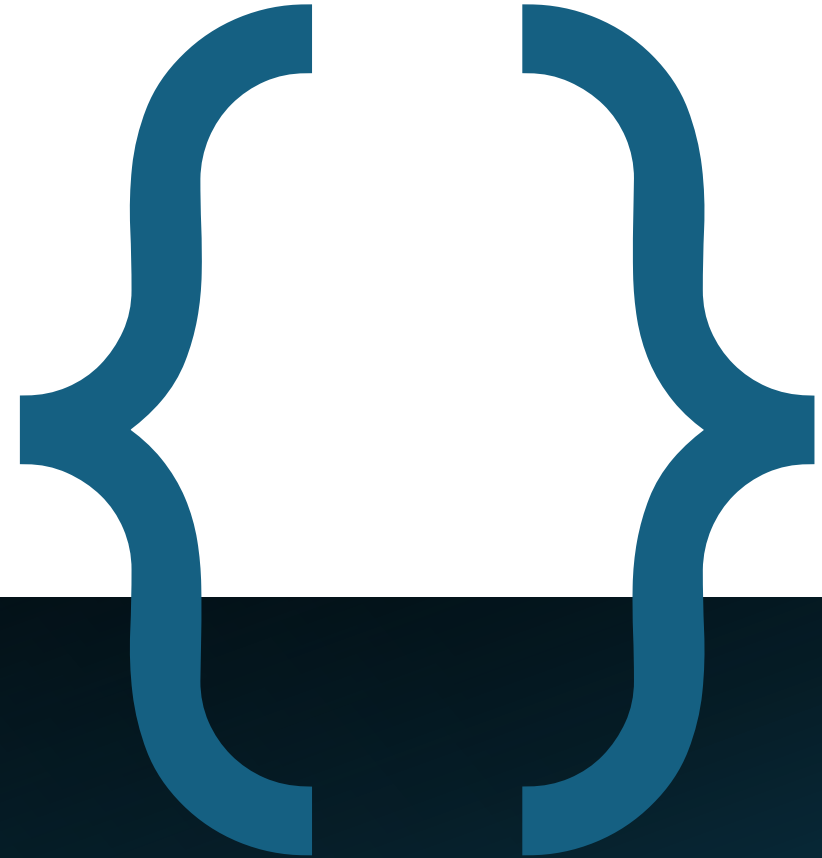


Binary Trees and Binary Search Trees in C++

Tree Structure, BST Construction, Traversals,
Applications, and Live Implementation

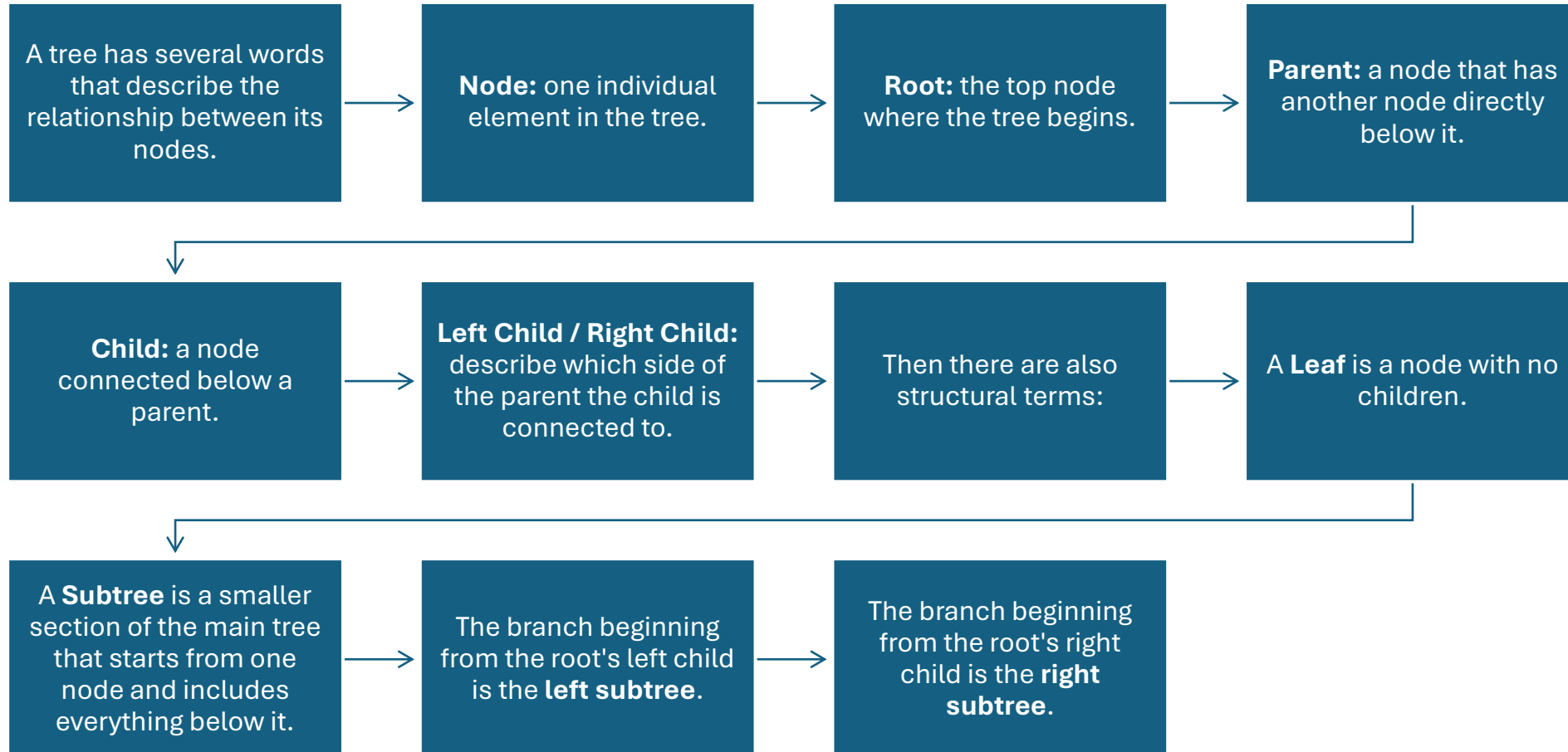
Presented by: Akash Patel
Instructor: Dr. Victor Govindaswamy



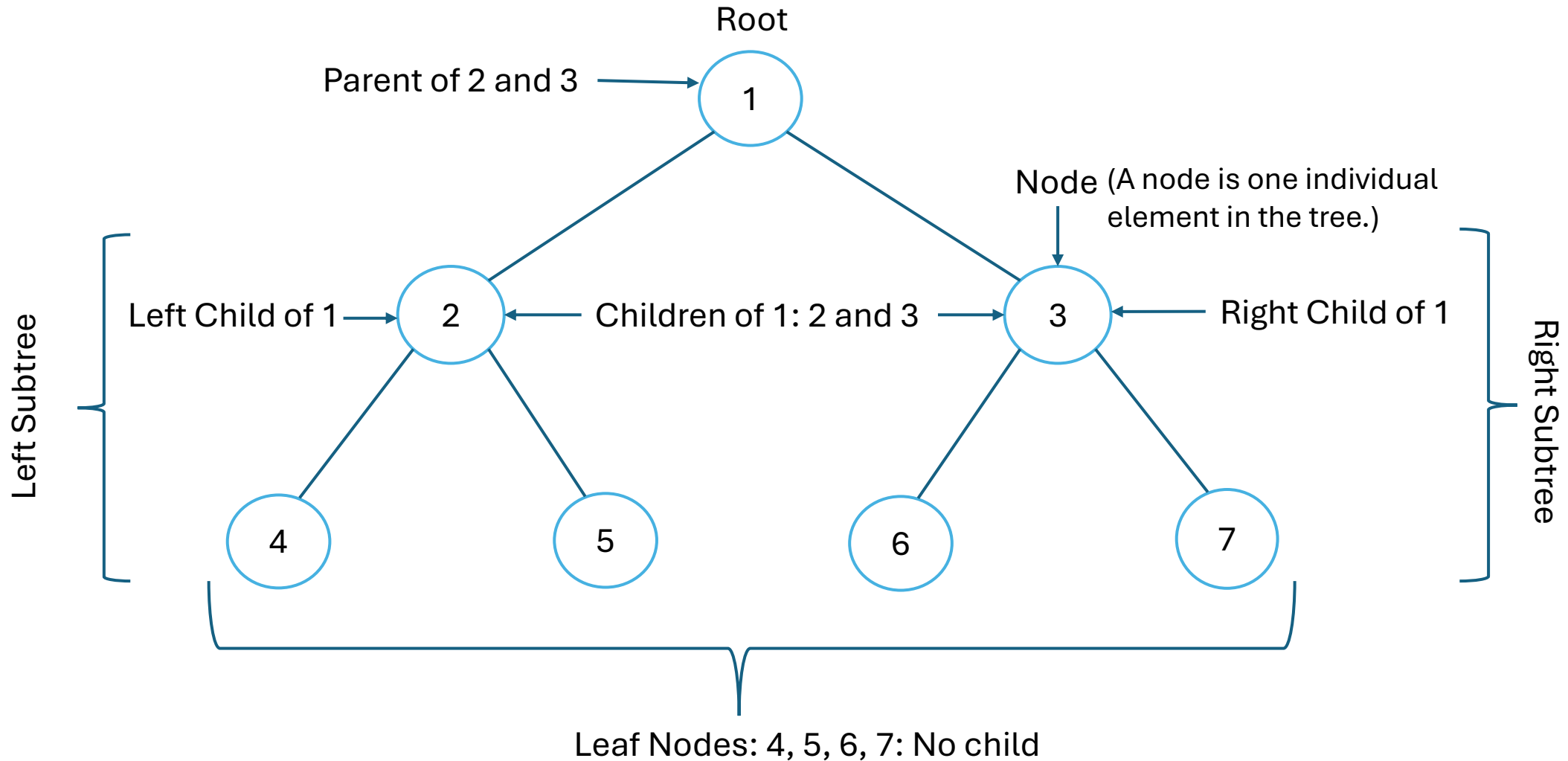
What Is a Binary Tree?

- A **Binary Tree** is a hierarchical data structure made of connected elements called **nodes**.
- Each node can have a maximum of **two children**:
 - one left child
 - one right child
- The first node at the top is called the **root**. From the root, the tree branches downward into smaller sections.
- A Binary Tree does **not automatically require values to be sorted**. A smaller value could appear on either side depending on how that particular tree is designed.
- This makes a Binary Tree useful for representing information that naturally has a parent-and-child relationship.
- **Simple idea:**
- Unlike an array, which stores information in a straight line, a tree organizes information in levels and branches.

Understanding Binary Tree Terminology



Binary Tree Terminology



How a Tree Node Is Represented in C++

- `class Node { public: int data; Node* left; Node* right; };`
- **int data**
 - Stores the actual value of the node.
 - For example, a node may store 48.
- **Node* left**
 - Stores the address of the node connected to the **left side**.
 - If there is no left child, the pointer can be `nullptr`.
- **Node* right**
 - Stores the address of the node connected to the **right side**.
 - Together, these two pointers allow nodes to connect and form branches.
- **Main idea:**
 - The value is stored in `data`, while `left` and `right` connect one node to other nodes.

What Is a Binary Search Tree (BST)?

- A **Binary Search Tree, or BST**, is a special type of Binary Tree that keeps values in an **ordered way**.
- Each node can have up to two children.
- Values **smaller** than the current node go to the **left side**.
- Values **larger** than the current node go to the **right side**.
- This ordering makes it easier to search for and organize values compared with a general Binary Tree.
- **BST Ordering Rule**
- **Smaller value → LEFT**
- **Larger value → RIGHT**
- For example, suppose the current node is:
- **48**
- 27 is smaller than 48, so it goes to the left.
- 73 is larger than 48, so it goes to the right.
- But insertion may require more than one comparison.
- For example, to insert 36:
- $36 < 48 \rightarrow$ move left to 27
- $36 > 27 \rightarrow$ move right
- Insert 36 there
- **Main idea:**
The program keeps comparing the new value with existing nodes until it finds an empty place where the value belongs.



Binary Tree vs. Binary Search Tree



■ Binary Tree

- Every node can have up to two children.
- There is no required numerical ordering rule.
- Its main purpose is to represent hierarchical relationships.

■ Binary Search Tree (BST)

- A BST is a **special type of Binary Tree**.
- Smaller values are placed in the **left subtree**.
- Larger values are placed in the **right subtree**.
- This ordering can make searching and maintaining sorted information easier.

■ Important point:

- Every BST is a Binary Tree, but not every Binary Tree is a BST.
- 

Our Dataset and Duplicate Values

- I used my own dataset:
- **48, 27, 73, 14, 36, 58, 91, 32, 42, 54, 36, 73**
- There are **12 original values**, which meets the assignment requirement.
- **Duplicate values**
- 36 appears twice.
- 73 appears twice.
- For this BST:
- **The first occurrence is inserted, but the repeated occurrence is ignored.**
- So the duplicate occurrences that are discarded are:
- **second 36**
- **second 73**
- This prevents the same value from being inserted twice into our BST.

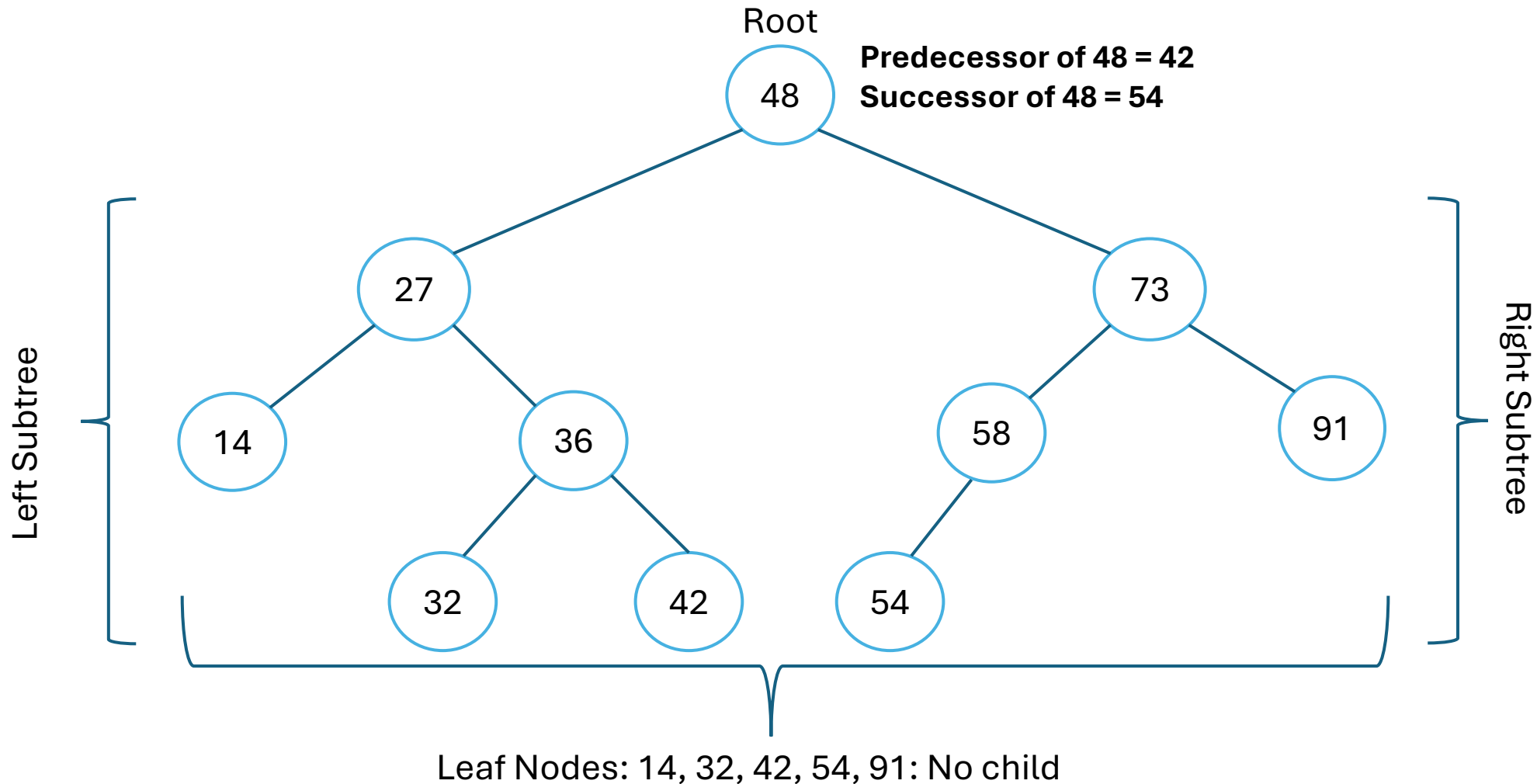
Completed Binary Search Tree (BST)

Original values:

48, 27, 73, 14, 36, 58, 91, 32, 42, 54, 36, 73

Duplicates: 36 and 73

Discarded duplicate occurrences: second 36 and second 73



What Is Tree Traversal?

- A tree contains several nodes, but we need a clear method to decide **which node should be visited first, second, third, and so on.**
- That process is called **tree traversal.**

- we use three traversal methods:
- **Inorder**
- **LEFT → ROOT → RIGHT**
- **Preorder**
- **ROOT → LEFT → RIGHT**
- **Postorder**
- **LEFT → RIGHT → ROOT**

- The tree itself does not change.
- **Only the order in which we visit its nodes changes.**
- This is an important point to explain in your video because beginners often think each traversal creates a different tree.



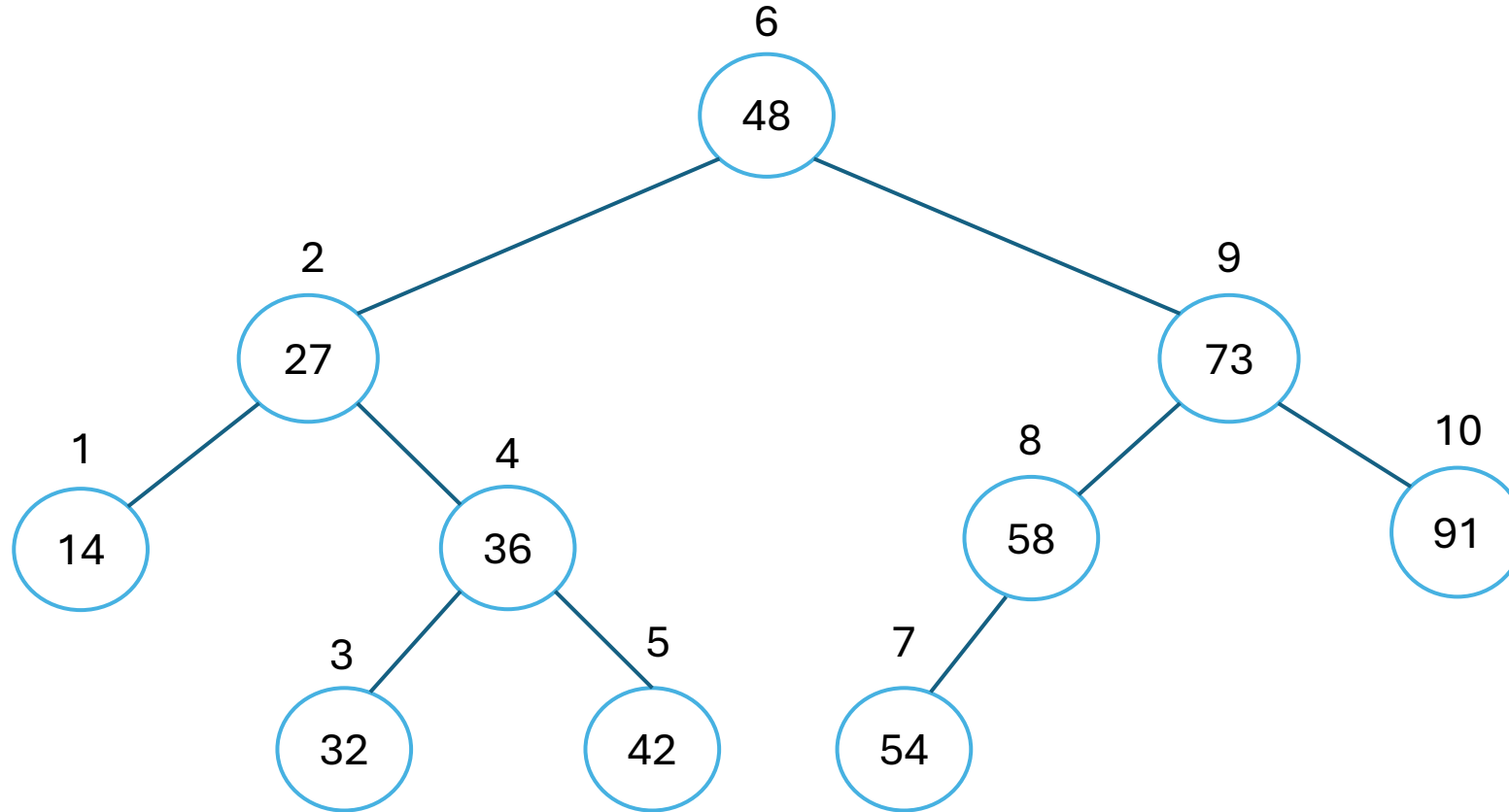
Understanding Inorder Traversal

- **Inorder follows: LEFT → ROOT → RIGHT**
- We begin by going as far as possible into the left subtree.
- Then:
- visit the current root,
- and finally visit the right subtree.
- For our BST, this produces:
- **14 → 27 → 32 → 36 → 42 → 48 → 54 → 58 → 73 → 91**
- **Why is this important?**
- Because a BST already stores:
- smaller values on the left,
- larger values on the right.
- So visiting:
- **LEFT → ROOT → RIGHT**
- naturally produces the numbers in **sorted ascending order**.



Inorder Traversal – LEFT → ROOT → RIGHT

➤ **Original values:** 48, 27, 73, 14, 36, 58, 91, 32, 42, 54, 36, 73

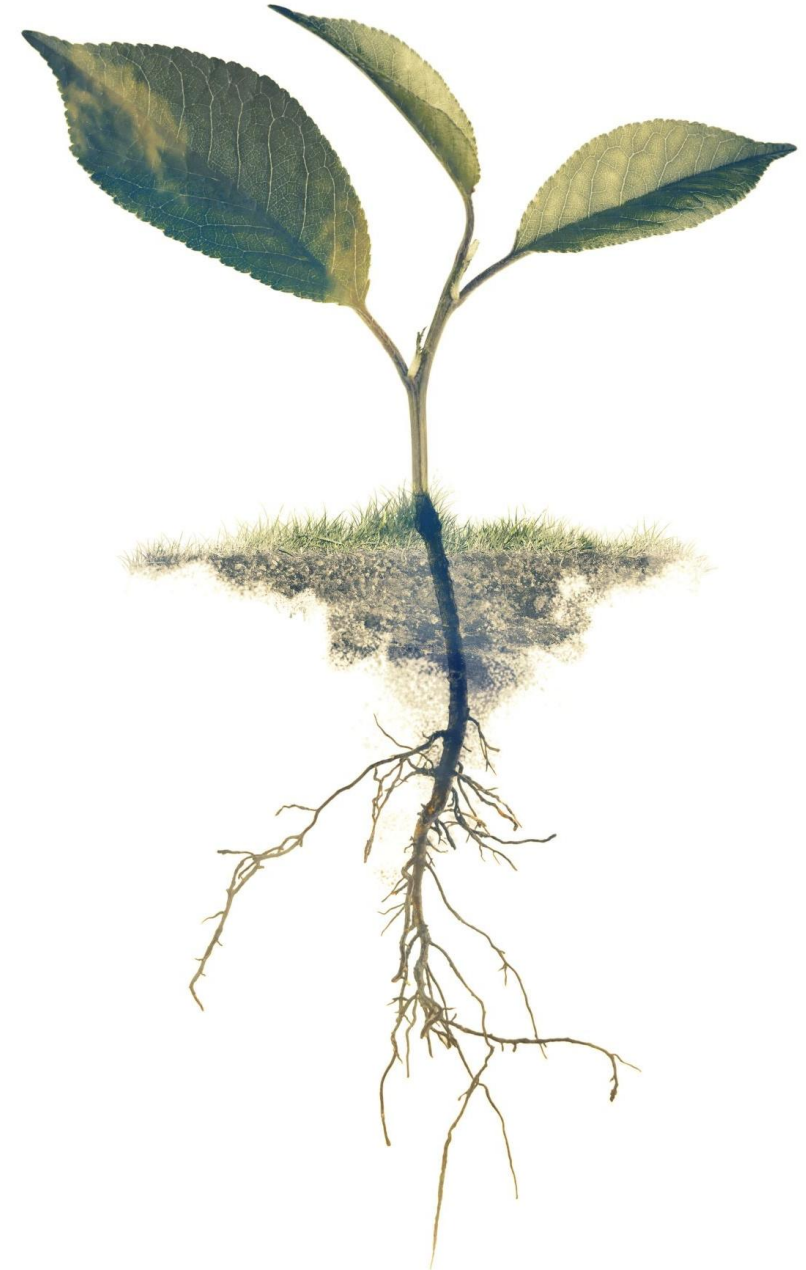


➤ **Inorder:** 14 → 27 → 32 → 36 → 42 → 48 → 54 → 58 → 73 → 91

➤ Inorder visits the left subtree first, then the root, and then the right subtree. In a BST, this gives the values in sorted order.

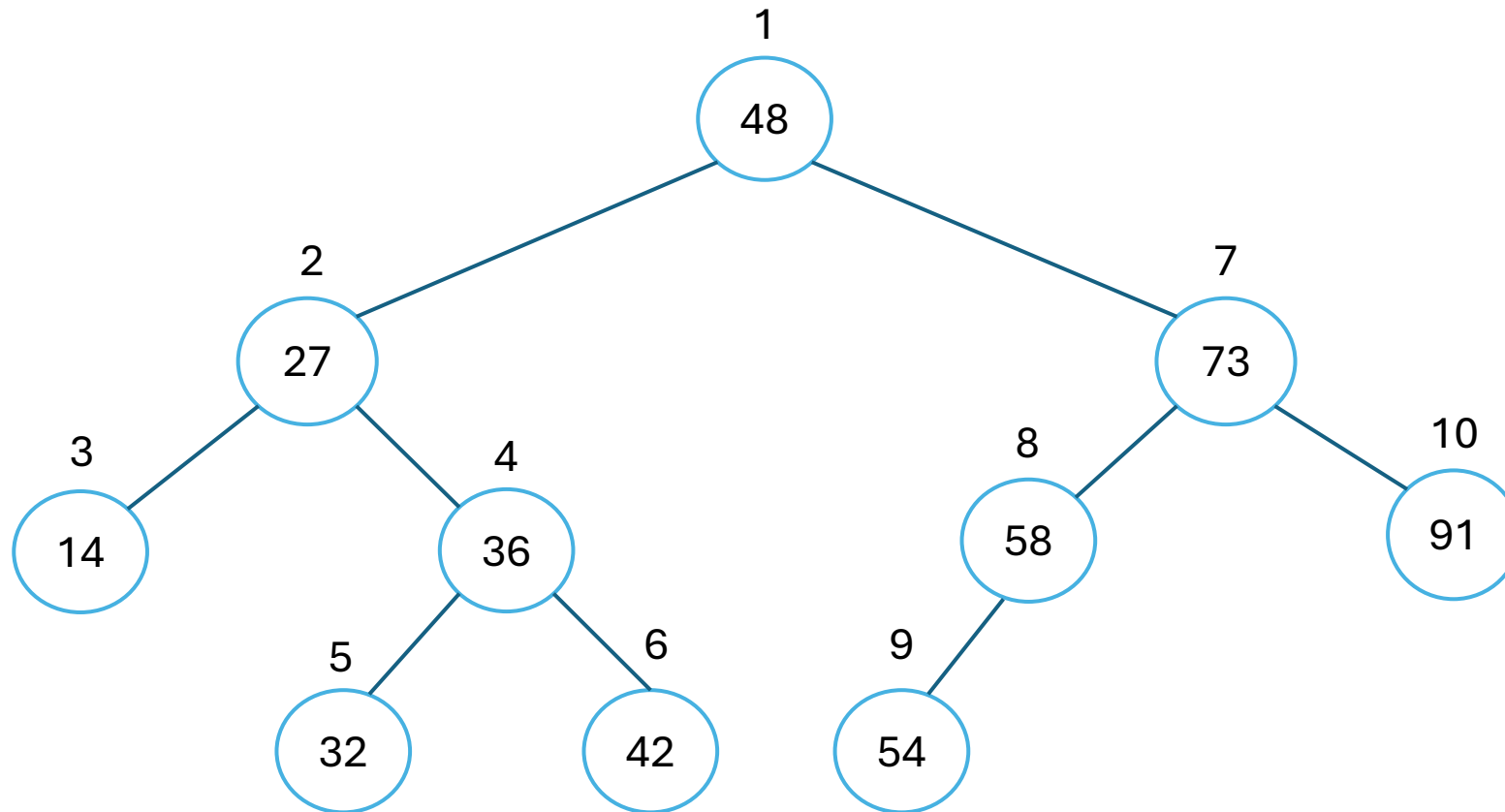
Understanding Preorder Traversal

- **Preorder follows: ROOT → LEFT → RIGHT**
- The main difference is that we visit the **parent/root first** before visiting its children.
- For our tree:
- Start at root 48.
- Then complete the entire left subtree beginning with 27.
- After the left side is finished, move to the right subtree beginning with 73.
- Our exact order becomes:
- **48 → 27 → 14 → 36 → 32 → 42 → 73 → 58 → 54 → 91**
- **Easy way to remember:**
- In preorder, the parent gets attention **before** its children.



Preorder Traversal – ROOT → LEFT → RIGHT

➤ **Original values:** 48, 27, 73, 14, 36, 58, 91, 32, 42, 54, 36, 73



➤ **Preorder:** 48 → 27 → 14 → 36 → 32 → 42 → 73 → 58 → 54 → 91

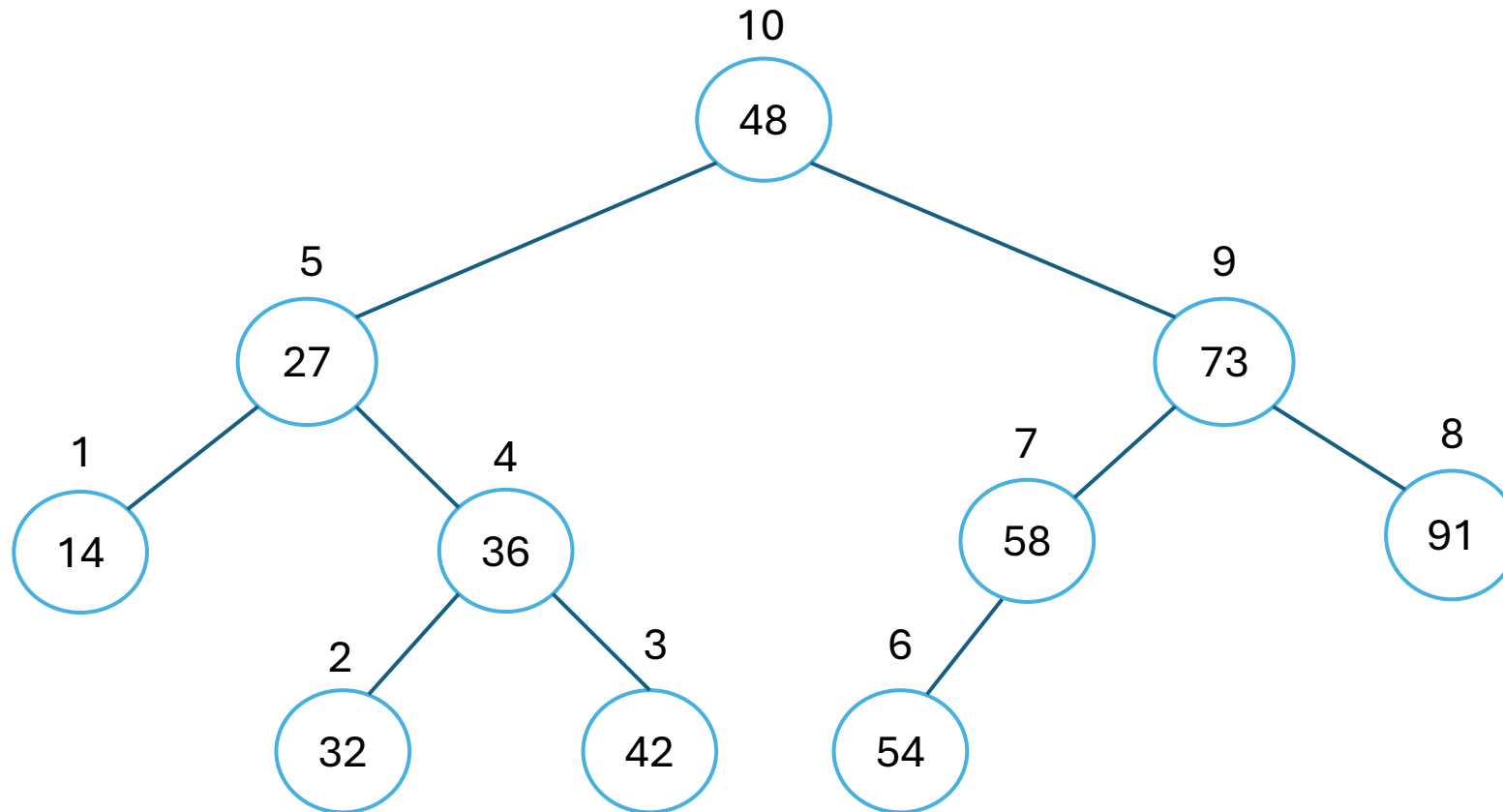
➤ Preorder visits the root first, then the left subtree, and then the right subtree. The main idea of preorder is that each parent node is visited before its children.

Understanding Postorder Traversal

- Postorder works differently:
- **LEFT → RIGHT → ROOT**
- This means the parent/root is visited **after both sides underneath it are finished**.
- For our tree:
- Complete the left subtree first.
- Then complete the right subtree.
- Only after both sides have been visited do we visit the root.
- The exact result is:
- **14 → 32 → 42 → 36 → 27 → 54 → 58 → 91 → 73 → 48**
- Notice that 48, which was first in preorder, becomes **last in postorder**.
- **Easy way to remember:**
- In postorder, children are handled before their parent.

Postorder Traversal – LEFT → RIGHT → ROOT

➤ **Original values:** 48, 27, 73, 14, 36, 58, 91, 32, 42, 54, 36, 73



➤ **Postorder:** 14 → 32 → 42 → 36 → 27 → 54 → 58 → 91 → 73 → 48

➤ Postorder traversal visits the left subtree first, then the right subtree, and visits the root node last. In this tree, we finish both sides before coming back to 48, so the root is the final node visited.

Real-World Uses of Trees

File and Folder Systems

A tree naturally represents folders, subfolders, and files because one folder can contain several smaller items underneath it.

Organization Structure

A company can be represented hierarchically:
CEO → managers → employees.

The parent/child relationship of a tree matches this type of organization very naturally.

HTML / XML Structure

Web documents contain elements inside other elements.

A tree helps represent which elements are parents and which ones are nested children.

Important: A File System Is a Tree, Not Necessarily a BST

A file system is usually represented as a **hierarchical tree**:

Folder

Documents

- Report.docx
- Notes.txt

Pictures

- photo1.jpg
- photo2.jpg

This works because folders can contain other folders and files.

However:

A file system does not normally need the BST rule of “smaller left, larger right.”

So:

File system → Tree structure
but not necessarily → Binary Search Tree

That distinction is important.

Where Binary Search Trees Are Useful

- A BST is useful when we want information to remain **ordered**.
- **Searching Ordered Information**
 - Because smaller and larger values are separated into different sides, we can follow comparisons while searching instead of checking values randomly.
- **Maintaining Sorted Data**
 - The BST rule keeps values organized according to their comparisons.
- **Indexing / Lookup Structures**
 - BST-style organization can help with situations where repeated searching and ordered access are important.
- **Why a BST fits these situations:**
 - Its structure is based on value comparisons, so the location of a value is meaningful rather than completely random.

What Happens When a Duplicate Is Entered?

- This gives duplicate handling its own natural explanation before the live code.
- Suppose the program tries to insert 36 again.
- During insertion:
 - Compare 36 with 48 → move left.
 - Compare 36 with 27 → move right.
 - Compare 36 with the existing 36.
- Now the program sees:
 - **new value == existing value**
 - Instead of creating another node, the program reports or ignores the duplicate.
- **Result:**
 - The BST remains unchanged.



From Diagrams to the C++ Program

- **Now I will demonstrate the Binary Search Tree program in C++.**
- During the live demonstration I will explain:
 - the Node class,
 - data, left, and right,
 - how the BST starts with an empty root,
 - how insertion compares values,
 - how duplicate values are detected,
 - and how recursive functions perform inorder, preorder, and postorder traversals.
- **After explaining the important parts of the code, I will compile and run the program and compare the program output with the traversal diagrams shown earlier.**

What I Learned from Trees and BSTs



At first, Binary Trees and BSTs looked similar to me, but I learned that the BST has an additional ordering rule.



Building the tree one value at a time helped me understand why every node ends up in a particular location.



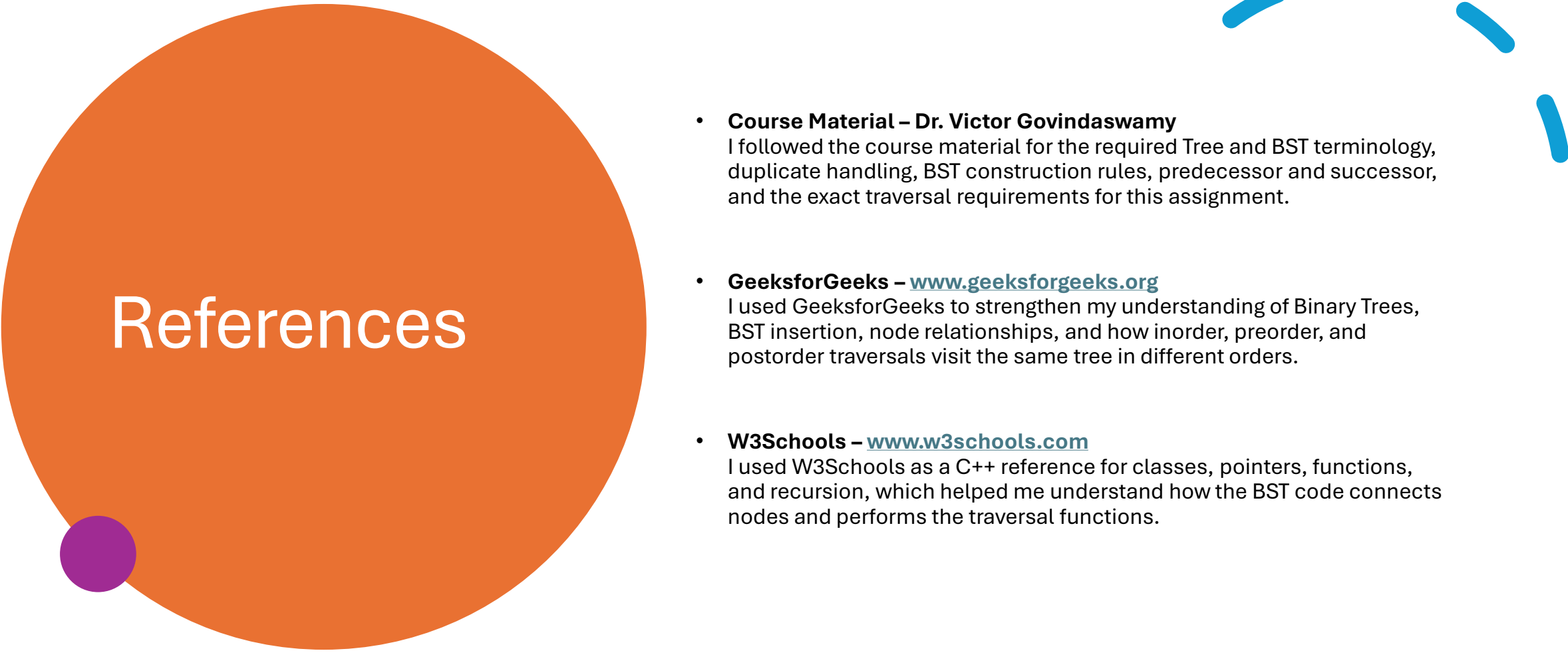
Traversals showed me that the **same tree can be read in three completely different orders** depending on the purpose.



I also understood why duplicate handling matters, because inserting repeated values can change how we define or manage the BST.



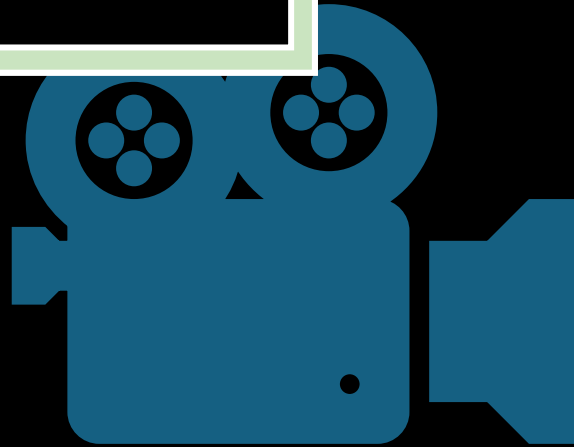
Connecting the diagrams with the C++ output made the traversal rules much easier for me to understand.



References

- **Course Material – Dr. Victor Govindaswamy**
I followed the course material for the required Tree and BST terminology, duplicate handling, BST construction rules, predecessor and successor, and the exact traversal requirements for this assignment.
- **GeeksforGeeks – www.geeksforgeeks.org**
I used GeeksforGeeks to strengthen my understanding of Binary Trees, BST insertion, node relationships, and how inorder, preorder, and postorder traversals visit the same tree in different orders.
- **W3Schools – www.w3schools.com**
I used W3Schools as a C++ reference for classes, pointers, functions, and recursion, which helped me understand how the BST code connects nodes and performs the traversal functions.

Thank You



- **Thank You for Watching!**
- **In this video I covered:**
 - Binary Trees
 - BST Rules
 - Tree Terminology
 - Node Structure
 - BST Construction
 - Duplicate Handling
 - Inorder
 - Preorder
 - Postorder
 - Applications
 - Live C++ Demonstration
- **I hope this video helped make Binary Trees and Binary Search Trees easier to understand. Thank you for taking the time to watch my explanation and demonstration.**
- **Presented by: Akash Patel**
- **Instructor: Dr. Victor Govindaswamy**
- **Thank you for your time and attention.**